

OOP

目录

OOP	1
○ 哇好神秘还有绪论课 2026-02-28	3
壹 编译源程序 2026-03-03	3
一、编译链接	3
二、extern 关键字	4
三、宏定义/常量	4
四、Make	4
五、函数重载：一个函数名字，两个以上实现方法。	5
六、auto, decltype	5
auto	5
decltype	5
七、内存申请和释放	6
贰 对象 2026-03-10	6
一、访问权限：	6
二、this 指针	7
三、内联函数	7
四、new delete	7
叁 构造函数和析构函数 2026-03-17	7
一、构造函数：诞生	7
委派构造函数：调用其他构造函数。	8
就地初始化：	8
默认/缺省构造函数：	8
对象数组的初始化：	9
二、析构函数：死亡	9
三、局部对象的构造和析构	9
四、全局对象的构造和析构	9
五、引用	9
六、运算符重载	11
前缀后缀重载区分	11
() 重载	11
[] 重载	11
流运算符重载	11
七、友元 2026-03-24	12
肆 静态变量和静态函数 常量	13
一、静态变量，静态函数	13
静态数据成员 / 类变量	13
静态成员函数	14
二、常量	14
常量数据成员	14
常量成员函数	14
常量对象	14
常量静态变量	14
再谈常量对象和静态对象：构造和析构	15
伍 参数对象的构造和析构	15
一、参数中的常量和常量引用	16
二、拷贝构造函数	17
拷贝构造函数：实例 1	17
拷贝构造函数：实例 2	18
问题？	19
三、内联函数、左值引用与右值引用	19

左值与右值	19
右值引用	19
引用绑定规则	20
右值引用示例	20
右值引用的意义	20
四、移动构造函数	21
移动构造函数 vs 拷贝构造函数	21
移动构造函数示例	21
移动构造函数实例	21
情况一：开启返回值优化（默认）	22
情况二：禁止返回值优化 + 有移动构造函数	22
情况三：禁止返回值优化 + 删除移动构造函数	22
什么时候调用移动构造函数？什么时候调用拷贝构造函数？	23
拷贝构造函数的调用时机	23
移动构造函数的调用时机	23
返回值优化（RVO）	23
std::move 函数	23
构造函数综合实例	24
十、拷贝赋值运算符与移动赋值运算符	26
拷贝赋值运算符	26
拷贝赋值运算符实例	26
移动赋值运算符	26
拷贝/移动赋值运算符的调用时机	27
编译器自动合成的函数/运算符	27
十一、类型转换	27
类型转换概述	27
方法一：类型转换运算符	28
方法二：类型转换构造函数	28
两种方法的使用	28
类型转换运算符：错误示例	29
类型转换实例	29
禁止自动类型转换	30
强制类型转换	31
强制类型转换示例	31
陆 组合与继承 2026-04-07	31
一、对象(类)之间的关系	31
二、组合	32
对象组合示例	32
子对象构造	32
对象组合示例：构造与析构	32
对象组合运行结果	33
隐式定义的拷贝构造与赋值运算	33
对象组合示例：拷贝与赋值	34
三、继承	35
基本概念	35
什么不能被继承？	35
继承示例	35
派生类对象的构造与析构过程	36
调用基类构造函数：隐式调用默认构造	36
调用基类构造函数：显式调用	36
继承基类构造函数（using）	37
继承基类构造函数（多个构造函数）	37
继承基类构造函数（注意事项）	37
如何选择继承方式？	38

public 继承	38
private 继承	38
成员访问权限	38
公有继承：基类公有成员的访问	38
私有继承：基类公有成员的访问	38
私有继承：用 using 打开基类公有成员的访问权限	39
私有继承：基类私有、保护成员访问	39
基类成员访问权限与三种继承方式	40
组合与继承的对比	40
组合示例：has-a	40
继承示例：is-a	41
四、重写隐藏与重载	41
重载（overload）	41
重写隐藏（redefining）	42
函数重写隐藏示例	42
恢复基类成员函数示例	42
using 关键字总结	43
五、多重继承	43
多重继承的问题	43
多重继承示例	43
六、课后练习	44

○ 哇好神秘还有绪论课 2026-02-28

我去，居然是雷霆方法论基础？！结构化？！面向对象？！神秘秘神？！

不能反直觉反常识？！强强！？ 对象？我的对象在哪里？！弱弱,,， **属性和操作？！静态和动态？！抽象？！**
过程抽象和数据抽象？！eg.类型是什么？！**数据的存储和表示+数据支持的操作？！**

壹 编译源程序 2026-03-03

一、编译链接

举例：argv argc 等参数的运用

实际意义上的一个完整编译流程如下：

```
g++ -c ex1.cpp
g++ -o ex1 ex1.o
```

第一步是编译，第二步是链接。注意到还有多文件链接。

```
g++ -c ex1.cpp ex1.o
g++ -c func1.cpp func1.o
g++ ex1.o func1.o -o ex1
```

当然我们平时偷懒是可以的。只是想告诉你这个编译和链接的实际过程，第二步把两个文件里的函数链接了？！

链接文件里面声明和定义不相同会类型报错。 #include 的本质是把代码直接复制下来了。**声明可以多声明，定义只能定义一次。** 声明的时候可以修改只说参数类型，不喊名字。**变量也可以声明或者定义。即使不赋值也可以被定义！** 注意下面的例子：

```
// num.cpp
int a=1;
```

```
// ex1.cpp
int a;
int main(){
```

```

    a=1;
    return 0
}

```

这时就炸了。重复定义。 **定义=声明+内存分配**

二、extern 关键字

有一点点烧脑。用于全局变量的共享。

```

//num.h
extern int a;
extern int b;
extern int c;
extern int d[10];

//num.cpp
#include<num.h>
a=1;
b=2;
c=3;

```

头文件中只声明，不定义，目的是防止重复定义。

三、宏定义/常量

请使用 const 以保证类型的正确性！对于宏：

```

#define name(value) command_string
#define sqr(x) ((x)*(x))

sqr(3+2) = 25

```

请注意这个时候括号十分重要。因为不带括号的时候只是把串原封不动地搬运过来。宏定义可以用来防止头文件被反复包含。

```

// Method 1
#ifndef _HEAD_
#define _HEAD_
...(如声明/定义)
#endif
// Method 2
#pragma once

```

还可以用来神秘的 debug。

```

#define DEBUG
#ifdef DEBUG
debug();
#endif

```

四、Make

Makefile 编写规则：只编译并链接被修改的文件，其他不管。但是如果头文件被改变了，所有引用了这个头文件的都要重新编译。格式举例：

```

all: main test
main: main.cpp stu.cpp
    g++ -o main main.cpp stu.cpp

```

```
test: stu.cpp stu_test.cpp
    g++ -o test stu_test.cpp stu.cpp
clean:
    rm main test

//bash 举例
make all
make test
make main
make clean
```

Make 的更多用法还是自己去查吧。氧气和臭氧优化后不能 gdb。

五、函数重载：一个函数名字，两个以上实现方法。

关键在于参数类型的不同。但是如果出现了参数的缺省，就会产生二义性。

举例：

```
int fun(int a, int b=0);
int fun(int a);
```

会产生二义性。因为当调用 fun(1)的时候，编译器无法判断是调用第一个函数还是第二个函数。

但是！

```
int fun(int a);
int fun(float a);
```

就不会产生二义性。因为当调用 fun(1)的时候，编译器会选择第一个函数，因为参数类型是 int，而不是 float。

六、auto, decltype

auto 和 decltype 都是 C++11 引入的类型推断机制，可以编译器根据上下文自动推断变量的类型。

auto

```
auto x = 5; // x 的类型被推断为 int
auto y = 3.14; // y 的类型被推断为 double
auto z = "Hello"; // z 的类型被推断为 const char*
```

- auto 必须在定义时初始化。
- auto 不能用于函数参数声明。
- auto 不能用如 sizeof, typeid 等上下文中。
- 在定义模板函数时用于声明依赖模板参数的返回类型：

```
template<typename _Tx, typename _Ty>
void Multiply(_Tx x, _Ty y){
    auto v = x * y; // 返回类型由编译器根据表达式 x * y 推断
    std::cout << v << std::endl;
}
Multiply(3, 4.5); // 输出 13.5, v 的类型被推断为 double
```

decltype

- 重用匿名类型：

```
struct {
    int a;
    double b;
} myStruct;
decltype(myStruct) anotherStruct; // anotherStruct 的类型与 myStruct 相同
```

除此之外还可以用来追踪推导 auto 返回的类型，在 C++ 14 后不必要：

```
auto func(int x, int y) -> decltype(x+y) {
    return x + y; // 返回类型被推断为 int
}
```

七、内存申请和释放

```
int* p = new int(10); // 申请一个 int 类型的内存
int *arr = new int[5]; // 申请一个 int 类型的数组
delete p; // 释放单个 int 类型的内存
delete[] arr; // 释放数组内存
```

nullptr 是 C++11 引入的空指针常量，表示一个空指针。使用 nullptr 可以避免与整数 0 混淆，提高代码的可读性和安全性。

贰 对象 2026-03-10

成员函数必须在类的内部声明，但是可以在类的外部定义（实现）。

举例：

```
class MyClass {
public:
    void myFunction(); // 成员函数声明
};
void MyClass::myFunction() { // 成员函数定义
    // 函数体
}
```

一、访问权限：

- public：公共成员，类的外部可以访问。
- private：私有成员，只有类的内部可以访问。也是默认访问权限。
- protected：受保护成员，只有类的内部和派生类可以访问。

可以通过 object.member 的方式访问对象的成员函数和成员变量。或者 pointer->member 的方式访问指针指向对象的成员函数和成员变量。在类外只能访问 public 成员。

神秘：在类的内部可以访问传入的同一个类的对象的 private 成员！因为它们属于同一个类。

举例：

```
class MyClass {
private:
    int privateVar;
public:
    void setPrivateVar(int value) {
        privateVar = value; // 访问自己的 private 成员
    }
    void accessOther(MyClass& other) {
        other.privateVar = 10; // 访问同类的其他对象的 private 成员
    }
};
```

二、this 指针

指向当前对象的指针。可以在成员函数内部使用 this 来访问当前对象的成员变量和成员函数。

举例：

```
class MyClass {
private:
    int value;
public:
    MyClass(int v) : value(v) {}
    void printValue() {
        cout << "Value: " << this->value << endl;
    }
};
```

错综复杂的类外类内关系.....阿弥诺斯，小白手套欸呦喂，我这不赖，已取餐。

三、内联函数

由于和宏代码是直接拷贝复制代码到指定位置，会产生莫名其妙的边际效应，你永远无法想象到各种优先级问题，即使加了括号也不一定能完全解决。

相比之下，**内联函数**可以执行类型检查，可调试。它的本质是生成和函数等价的表达式。并且**宏定义**无法调用私有成员函数，而内联函数可以。

注意事项：

- 避免大段代码的内联，因为会增加编译时间和可执行文件的大小。
- **避免对包含循环、递归或复杂逻辑的函数使用内联。**
- 避免将声明和定义分开，因为内联函数必须在调用之前定义。
- 定义在类内部的成员函数默认是内联的，但这只是一个建议，编译器可能会选择不内联。
- **一般构造函数、析构造函数**通常会被编译器自动内联。

inline **永远只是建议修饰，不是命令**。编译器有权拒绝不合理的内联请求。

四、new delete

new 生成一个类对象，并且返回它的指针。、

```
A *pA = new A(1);
```

delete 释放一个类对象。

叁 构造函数和析构造函数 2026-03-17

回顾：要对用户定义类型进行严格检查；要隐藏属性；要自动化对象的初始化和清除，做好对象内存分配等等。

一、构造函数：诞生

- 构造函数没有返回值
- 构造函数名与类名相同
- 构造函数可以重载

举例：

```
class Student {
    int ID;
public:
    Student(int id) : ID(id) {}
    Student(int year, int order){
        ID = year * 10000 + order;
    }
};
```

注意！按照声明顺序而不是初始化顺序排列成员！ 如下面 ID1 不可预测。

```
class Student {
    int ID1;
    int ID2;
public:
    Student(int id) : ID2(id), ID1(ID2) {}
};
```

委派构造函数：调用其他构造函数。

```
class Info{
public:
    Info(int a):a(a){}
    Info(int a,int b):Info(a){b=b;}
    Info(int a,int b,int c):Info(a,b){c=c;}
};
```

就地初始化：

```
class A{
private:
    int a{0};
    double b=2.0;
public:
    A(){}
};
```

默认/缺省构造函数：

```
class A{
public:
    A(){};
};

A a;
A b=A();
A c();
// 这个时候 c 是一个函数声明，而不是一个对象声明。它可以生成一个对象。

//显式默认构造函数
class A{
public:
    A()=default;
};

//显式删除默认构造函数
class A{
public:
    A()=default;
    A(int i){}
    A(char cls)=delete;
};

A a('c');
```

一个实例生成的时候，会先调用成员的构造函数（如果成员没有就默认构造），再构造自己的。

如果你已经定义了构造函数，编译器不会帮你默认构造了！！！！

对象数组的初始化:

```
A a[50];  
// 只有一个参数  
A a[3]={1,2,3};  
// 多个参数  
A a[3]={A(1,2),A(3,4),A(5,6)};
```

二、析构函数：死亡

- 没有参数
- 唯一
- 没有返回值

举例：

```
class MyClass {  
    int num;  
    int* ID_list;  
public:  
    MyClass() { // 构造函数  
        // 构造函数体  
    }  
    ~MyClass() { // 析构函数  
        // 析构函数体  
        if(ID_list) delete[] ID_list; // 释放内存  
    }  
};
```

析构函数会在对象生命周期结束时自动调用。比如说程序结束的时候：

```
//我们此前 Test 里面创建了一个 Member m  
  
Test t;  
int main(){return 0;}  
// 输出: ~Test() ~Member()
```

晚构造，先析构！！

析构函数也会自动隐式定义。但是不会删除指针成员，因此可能导致内存泄漏。

三、局部对象的构造和析构

其实没啥好说的，就是看括号里面的东西。

四、全局对象的构造和析构

- 在 main() 之前构造
- 在同一编译单元中，按照定义顺序构造
- 在不同编译单元中顺序不确定
- 在 main() 执行完 return 语句后析构

尽量少用全局对象！ 全局对象之间最好不要有依赖关系，否则析构顺序会出问题。

五、引用

引用是对象的别名，可以理解为指针常量。引用必须在声明时初始化，并且一旦初始化就不能改变。

举例：

```
int a = 5;  
int& ref = a; // ref 是 a 的引用
```

```
ref = 10; // 修改 ref 也会修改 a
cout << a << endl; // 输出 10
```

二者绑定，改了一个另一个也会变。而且不能解绑。

有些时候和指针很像。

举例：交换两个变量的值

```
void swap(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}
```

看一个做错的题：

```
class Int {
public:
    int data;
    Int() { data = 1; }
    Int(int i): data(i) {}
};

void func1(Int& a, Int b) {
    a.data += b.data;
}

Int& func2(Int& a, Int b) {
    func1(a, b);
    Int tmp(a.data + b.data);
    return tmp;
}

int main() {
    Int a, b(3);
    Int& f = func2(a, b);
    cout << a.data << " ";
    cout << f.data << endl;
    return 0;
}
```

初始状态：a.data = 1（默认构造），b.data = 3（值构造）。

调用 func2(a, b) 时，Int& a 是 main 中 a 的引用，Int b 是值拷贝。

1. func1(a, b) 执行 a.data += b.data，即 $1 + 3 = 4$ ，main 中的 a.data 变为 4。
2. Int tmp(a.data + b.data) 在栈上构造局部变量，tmp.data = $4 + 3 = 7$ 。
3. return tmp 返回 tmp 的引用。

func2 返回类型为 Int&，而 tmp 是栈帧上的局部变量。函数返回后栈帧销毁，tmp 随之消亡，main 中的 f 成为悬空引用（dangling reference），指向已释放的内存。

注意：tmp 并非“返回前被销毁”，而是函数返回后随栈帧一起消亡。引用被传递出去了，但其所指向的对象已不复存在，后续访问是未定义行为。

引用的用途：

- 作为函数参数，可以避免复制对象，提高效率。
- 作为函数返回值，可以返回引用，而不是复制对象。
- 作为类的成员变量，可以简化代码，提高可读性。

- 相对指针，引用更安全，不会出现空指针的情况。

六、运算符重载

举例：

```
A& operator+=(A& a){
    data+=a.data;
    return *this;
}
```

可以重载的运算符：

- 算术运算符：+、-、*、/、%
- 关系运算符：==、!=、<、>、<=、>=
- 逻辑运算符：&&、||、!
- 位运算符：&、|、^、~、<<、>>
- 单目运算符：+、-、*、&、!
- 自增自减运算符：++、--
- 赋值运算符：=、+=、-=、*=、/=、%=、&=、|=、^=、<<=、>>=
- 空间运算符：new、delete、new[]、delete[]
- 其他运算符：()、[]、->、,、->*

前缀后缀重载区分

后缀关键是引入了哑元，哑元可以没有名，所以可能实际不会被调用。

```
A operator++(int){
    A temp(data);
    ++data;
    return temp;
}
```

() 重载

```
int operator()(int a, int b){
    return a+b;
}
```

[] 重载

```
int& operator[](int i){
    return data[i];
}
```

如果是引用，则可以修改原对象。否则只能读取。

注意：=, [], (), -> 这几个运算符只能重载为成员函数。否则可能会对是否自动合成重载符产生影响。

流运算符重载

```
ostream& operator<<(ostream& os, const A& a){
    os<<a.data;
    return os;
}

istream& operator>>(istream& is, A& a){
    is>>a.data;
    return is;
}
```

此时我们不得不谈到：友元。

七、友元 2026-03-24

```
#include <iostream>
using namespace std;

class Test {
    int id;
public:
    Test(int i) : id(i) { cout << "obj_" << id << " created\n"; }

    friend istream& operator>> (istream& in, Test& dst);
    friend ostream& operator<< (ostream& out, const Test& src);
};

istream& operator>> (istream& in, Test& dst) {
    in >> dst.id;
    return in;
}

ostream& operator<< (ostream& out, const Test& src) {
    out << src.id << endl;
    return out;
}

int main() {
    Test obj(1);
    cout << obj; // operator<<(cout,obj)
    cin >> obj; // operator>>(cin,obj)
    cout << obj;
    return 0;
}
```

友元是这个对象的“朋友”，可以访问这个对象的私有成员 / 保护成员。

举例：

```
class A{
    int data;
    friend B b;
    b.id=data;
}
```

友元函数举例：

```
class A{
    int data;
    friend void func(A a);
    friend void X::foo(A);
    friend X::X(Y),X::~~X();
};
A a;
func(a);
```

友元函数在类外定义，但是可以访问类的私有成员。且在 private,public 声明没有区别。

更详细的例子：

```

class Y; // 前向声明
class X
{
    int data;
    friend void func(X &x, Y &y);
};
class Y
{
    int data;
    friend void func(X &x, Y &y);
};
void func(X &x, Y &y)
{
    cout << x.data << y.data << endl;
}

```

友元类:

```

class Y {};
class A{
    int data;
    friend class Y;
    friend X;
}
class X{};
// X,Y 都可以访问 A 的私有成员

```

注意:

- 友元关系是**非对称的**，即 A 是 B 的友元，B 不是 A 的友元。
- 友元关系**不传递**，即 A 是 B 的友元，B 是 C 的友元，A 不一定是 C 的友元。
- 友元关系**不具继承性**，即 A 是 B 的友元，C 继承自 B，A 不一定是 C 的友元。
- 友元声明不能直接在里面定义了。

肆 静态变量和静态函数 常量

一、静态变量，静态函数

内部可链接，只能初始化一次，而且作用域仅限其声明的文件。**比如**在其他文件被 extern 声明后**不**可以在其他文件中使用。

前缀是 static。

静态数据成员 / 类变量

在整个类内所有对象共享，初始化时需要加 static。应该在 .h 文件中声明，在 .cpp 文件（实现文件）中定义，防止重复定义。

在实例化所有对象之前就已经分配了静态数据成员的内存空间。

举例:

```

// Test.h
class Test {
public:
    static int count;
    Test();
    ~Test();
};

// Test.cpp

```

```

#include "Test.h"
int Test::count = 0;
Test::Test() { count++; }
Test::~Test() { count--; }

// main.cpp
#include "Test.h"
int main() {
    Test a;
    Test b;
    cout << Test::count << endl; // 输出 2
    cout << a.count << endl; // 输出 2
    return 0;
}

```

静态成员函数

只能访问静态成员变量，不能访问非静态成员变量。因为非静态成员变量是每个对象独有的，而静态成员函数是所有对象共享的。

二、常量

- const 修饰的变量不能被修改。
- 修饰引用/指针时，引用/指针不能被修改，指向的内容不能被修改。
- 修饰函数返回值时，函数返回值不能被修改。

常量数据成员

可以在构造函数初始化列表中初始化、就地初始化，但是不能在构造函数体通过赋值初始化。

常量成员函数

不能修改非静态数据成员，即不能改变对象的状态。

常量对象

只能调用常量成员函数，不能调用非常量成员函数。

错误举例：

```

class Student {
    int ID;
public:
    Student(int id) : ID(id) {}
    int who() const { return ID; } // 常量成员函数
    int Who() { return ID; } // 编译错误，常量函数不能调用非常量成员
};

```

常量静态变量

需要在类外初始化。但是有两个例外：int 和 enum 可以就地初始化。

不存在常量静态函数。静态函数是所有对象共享的，而常量函数只能在常量对象上调用。

```

class Foo{
    static const int a=1; // 可以就地初始化
    static const int b; // 可以在类外初始化
    static const char* cs; // 不可以就地初始化
};

```

哦哦哦，给你看哥哥的大表格！

	静态数据成员	常量数据成员	常量静态数据成员 (除 int enum 外)	常量静态数据成员 (int, enum)
初始化				
就地初始化		✓		✓
初始化列表初始化		✓		
构造函数体内初始化				
类外初始化	✓		✓	✓
访问				
普通成员函数	✓	✓	✓	✓
静态成员函数	✓		✓	✓
常量成员函数	✓	✓	✓	✓
修改				
普通成员函数	✓			
静态成员函数	✓			
常量成员函数	✓			

再谈常量对象和静态对象：构造和析构

- 常量对象的构造和析构时机和普通对象一样。
 - 在 main() 函数开始执行前，全局对象被构造。
 - 在 main() 函数执行结束后，全局对象被析构。
 - 在执行到局部对象的代码前，局部对象被构造。
 - 在局部对象作用域执行结束后，局部对象被析构。
- 静态全局对象的构造和析构时机和普通全局对象一样。
- 函数中的静态对象在函数调用时构造，**离开作用域不析构**。第二次调用函数时，静态对象不会被重新构造，直接使用上一次构造的结果。在 main() 函数结束后，静态对象被析构。
- 类静态对象：类 A 的对象 a 作为类 B 的静态变量。
 - 在 main() 函数开始执行前初始化。
 - 在 main() 函数结束后析构。
 - 和 B 的构造和析构时机无关。

伍 参数对象的构造和析构

如果传递的是形参

```
class A{
public:
    const char* s;
    A(const char* str) : s(str) {
        cout << s << "A constructing" << endl;
    }
    ~A() {
        cout << s << "A destructing" << endl;
    }
};
```

如果传递的是引用

```
class A{
public:
    const char* s;
    A(const char* &str) : s(str) {
```

```

        cout << s << " A constructing" << endl;
    }
    ~A() {
        cout << s << " A destructing" << endl;
    }
};
void func(A a) {
    cout << a.s << "func" << endl;
}
int main() {
    A a("a");
    func(a);
    return 0;
}

```

输出：(构造一次，析构两次?！)

其实是 b 被拷贝构造函数进行了初始化。函数结束，b 被析构。

```

a A constructing
...
a A destructing
a A destructing

```

如果传递传递指针，你很有可能对同一块空间进行了两次释放，从而报错！

所以尽可能使用引用传递，避免拷贝构造函数和析构函数的调用，还可以节省时间开销。

语句	解析	调用次数
A *p = new A;	堆上创建单个对象	1
A p2[10];	对象数组（10 个元素）	10
A p3;	栈上普通对象	1
A *p4[10];	指针数组（不创建对象）	0
总计		12

- **类内初始化**：C++11 允许声明时直接赋值。
- **静态成员函数**：没有 this 指针，不能访问非静态成员。
- **静态数据成员**：必须类外定义，严禁在 .h 中直接定义（防重复定义）。
- **常量静态整型**：static const int 允许类内初始化。

核心警告：new[] 必须配对 delete[]。

$$\text{Actual Start Address} = pA - 4$$

若误用 delete pA:

1. 仅调用第一个元素的析构函数，导致内存泄漏。
2. 释放地址错误（应从 $pA - 4$ 处释放），直接导致程序崩溃。

一、参数中的常量和常量引用

最小特权原则：给足够完成任务的权限，但是不要多余修改、删除权限。

这个时候终于出现了我们喜闻乐见众所周知的：**常量引用**！

在这个例子里面，我们通过常量引用，将 const 修饰的变量传递给函数，这样函数内部就不能修改这个变量了。

```
void add(const int& a, const int& b)
```

二、拷贝构造函数

拷贝构造函数是 C++ 的一种特殊的构造函数，它用于创建一个新对象，该对象是已有对象的副本。

拷贝构造函数的格式如下：

```
ClassName(const ClassName& obj){id=obj.id;...}
```

其中，ClassName 是类的名称，obj 是一个已有的对象，用于初始化新对象。**用参数对象初始化当前对象！**

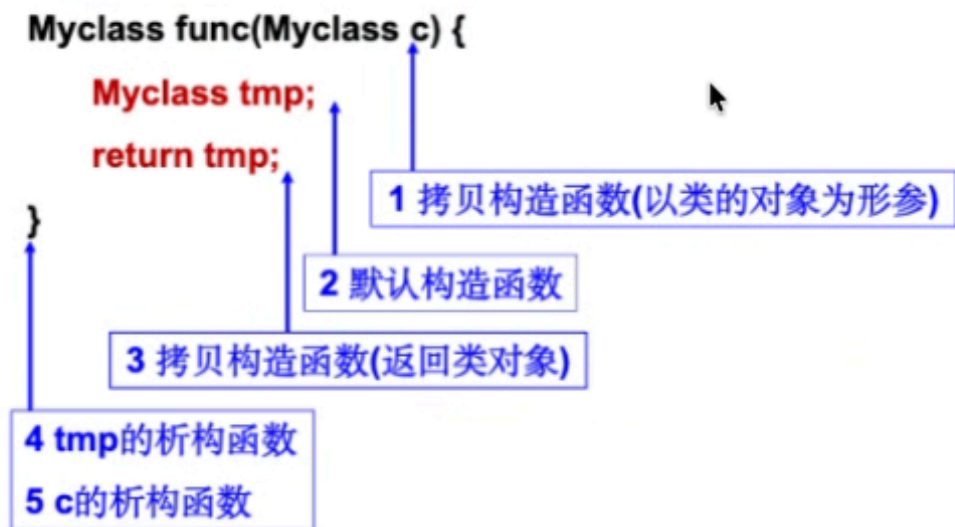
拷贝构造函数被调用的三种常见情况：

1. 用一个类对象定义另一个新的类对象 Test a; Test b(a); Test c=a;
2. 函数调用时以类的对象为形参 Func(Test a)
3. 函数返回类对象 Test Func(void)

编译器会自动调用“拷贝构造函数”，在已有对象基础上生成新对象。

拷贝构造函数：执行顺序

■ 以下述的func函数为例，调用该函数时，函数中各类构造函数和析构函数的执行顺序如下：



11

拷贝构造函数：实例 1

```
#include <iostream>  
using namespace std;  
class Test {  
public:  
    Test() { //构造函数  
        cout << "Test()" << endl;  
    }  
    Test(const Test& src) { //拷贝构造  
        cout << "Test(const Test&)" << endl;  
    }  
};
```

```

    }
    ~Test() { //析构函数
        cout << "~Test()" << endl;
    }
};
Test copyObj(Test obj) {
    cout << "func()..." << endl;
    return Test();
}
int main() {
    cout << "main()..." << endl;
    Test t;
    t = copyObj(t);
    return 0;
}

```

输出:

```

main()...
Test() //main函数内初始化 Test
Test(const Test&)//func参数 拷贝构造
func()...
Test() //初始化 Test类的对象
Test(const Test&) //返回时拷贝构造
~Test()
~Test()
~Test()
~Test()

```

注意: 采用编译选项禁止编译器进行返回值优化: `g++ test.cpp --std=c++11 -fno-elide-constructors -o test`

拷贝构造函数: 实例 2

```

#include <iostream>
#include <cstring>
using namespace std;
class Pointer {
    int *m_arr;
    int m_size;
public:
    Pointer(int i):m_size(i) { //构造
        m_arr = new int[m_size];
        memset(m_arr, 0, m_size*sizeof(int));
    }
    ~Pointer(){delete []m_arr;} //析构
    void set(int index, int value) {
        m_arr[index] = value;
    }
    void print();
};
void Pointer::print() {
    cout << "m_arr: ";
    for (int i = 0; i < m_size; ++ i) {
        cout << " " << m_arr[i];
    }
    cout << endl;
}
}

```

```
int main() {
    Pointer a(5);
    Pointer b = a; //调用默认的拷贝构造
    a.print();
    b.print();
    b.set(2, 3);
    b.print();
    a.print();
    return 0;
}
```

输出：

```
m_arr:  0, 0, 0, 0, 0  //a.print()
m_arr:  0, 0, 0, 0, 0  //b.print()
m_arr:  0, 0, 3, 0, 0  //b.print()
m_arr:  0, 0, 3, 0, 0  //a.print()
```

问题分析：位拷贝会使得对象 a, b 的指针成员 m_arr 指向同一个内存地址。当类内含指针类型的成员时，为避免指针被重复删除，不应使用隐式定义的拷贝构造函数。

问题？

造成程序效率显著下降。

尽可能避免使用拷贝构造函数，使用引用传递参数 !!!

解决方法：

- (1) 使用引用/常量引用传参数或返回对象；
- (2) 将拷贝构造函数声明为 private；
- (3) 用 delete 关键字让编译器不生成拷贝构造函数的隐式定义版本。

三、内联函数、左值引用与右值引用

左值与右值

首先区分什么是左值和右值：

- **左值：**可以取地址、有名字的值。例如变量 a。
- **右值：**不能取地址、没有名字的值。常见于常量、函数返回值、表达式结果。例如 1、a+b。

```
int a = 1;
int b = func();
int c = a + b;
// a, b, c 是左值
// 1, func()返回值, a+b的结果 是右值
```

左值可以取地址，并且可以被 & 引用（左值引用）：

```
int *d = &a;      // ✓ 左值可以取地址
int &e = a;       // ✓ 左值引用绑定左值
int *f = &(a+b);  // ✗ 右值不能取地址
int &g = a + b;   // ✗ 普通左值引用不能绑定右值
```

右值引用

虽然右值无法取地址，但可以被 && 引用（右值引用）：

```
int &&e = a + b; // ✓ 右值引用绑定右值
int &&f = a;     // ✗ 右值引用不能绑定左值
const int &g = 3; // ✓ 常量左值引用可以绑定右值
```

引用绑定规则

	非常量左值	常量左值	右值
非常量左值引用 &	✓		
常量左值引用 const &	✓	✓	✓
右值引用 &&			✓

关键点：

1. 左值引用能绑定左值，右值引用能绑定右值
2. **例外**：常量左值引用也能绑定右值（为了兼容性设计）
3. 所有的引用（包括右值引用）本身都是左值

右值引用示例

```
void ref(int &x) {
    cout << "left " << x << endl;
}
void ref(int &&x) {
    cout << "right " << x << endl;
}
int main() {
    int a = 1;
    ref(a);    // 输出：left 1 (a是左值，调用左值引用版本)
    ref(2);    // 输出：right 2 (2是右值，调用右值引用版本)
    return 0;
}
```

一个有趣的例子：

```
void ref(int &x) { cout << "left " << x << endl; }
void ref(int &&x) {
    cout << "right " << x << endl;
    ref(x);    // 这里调用哪个？
}
int main() {
    ref(1);    // 1是右值
    return 0;
}
```

输出：

```
right 1
left 1
```

因为**所有的引用本身都是左值**！x 虽然是右值引用参数，但它本身是个有名字的变量，所以是左值。

右值引用的意义

右值引用的核心用途：**延续即将销毁变量的生命周期**。

当函数返回一个临时对象时，这个对象马上就要被销毁。如果使用拷贝构造函数，需要重新开辟内存并复制数据，效率低下。使用右值引用配合移动构造函数，可以直接“偷走”临时对象的资源。

简单理解：

- 左值 = 有名字、有地址的“常住居民”，可以长期使用
- 右值 = 没名字、即将消亡的“临时工”，资源可以被“偷走”
- 左值引用 &= 给“常住居民”起别名
- 右值引用 &&= 在“临时工”消失前，把它的资源转移出来

四、移动构造函数

移动构造函数 vs 拷贝构造函数

```
// 拷贝构造函数
ClassName(const ClassName& VariableName);

// 移动构造函数
ClassName(ClassName&& VariableName);
```

核心区别：

- **拷贝构造**：重新开辟内存，复制数据
- **移动构造**：直接“偷走”临时对象的指针，不复制数据

	拷贝构造函数	移动构造函数
参数类型	const ClassName&	ClassName&&
内存操作	重新开辟并拷贝	直接复用指针
适用场景	对象需要长期保留	临时对象即将销毁

移动构造函数示例

```
class Test {
public:
    int *buf; // only for demo
    Test() {
        buf = new int[10]; // 申请一块内存
        cout << "Test(): this->buf @ " << hex << buf << endl;
    }
    ~Test() {
        cout << "~Test(): this->buf @ " << hex << buf << endl;
        if (buf) delete[] buf;
    }
    // 拷贝构造函数
    Test(const Test& t) : buf(new int[10]) {
        for(int i=0; i<10; i++)
            buf[i] = t.buf[i]; // 拷贝数据
        cout << "Test(const Test&) called. this->buf @ "
            << hex << buf << endl;
    }
    // 移动构造函数
    Test(Test&& t) : buf(t.buf) { // 直接复制地址，避免拷贝
        cout << "Test(Test&&) called. this->buf @ "
            << hex << buf << endl;
        t.buf = nullptr; // 将t.buf改为nullptr，使其不再指向原来内存区域
    }
};
```

移动构造函数实例

```
Test GetTemp() {
    Test tmp;
    cout << "GetTemp(): tmp.buf @ " << hex << tmp.buf << endl;
```

```

    return tmp;
}
void fun(Test t) {
    cout << "fun(Test t): t.buf @ " << hex << t.buf << endl;
}
int main() {
    Test a = GetTemp();
    cout << "main() : a.buf @ " << hex << a.buf << endl;
    fun(a);
    return 0;
}

```

情况一：开启返回值优化（默认）

编译指令：g++ test.cpp --std=c++11 -o test

```

Test(): this->buf @ 0x7fa908c04b90
GetTemp(): tmp.buf @ 0x7fa908c04b90
main() : a.buf @ 0x7fa908c04b90
Test(const Test&) called. this->buf @ 0x7fa908c04ba0
fun(Test t): t.buf @ 0x7fa908c04ba0
~Test(): this->buf @ 0x7fa908c04ba0
~Test(): this->buf @ 0x7fa908c04b90

```

注意：编译器进行了返回值优化（RVO），所以没有调用移动构造函数，也少调用了几次拷贝构造函数！

情况二：禁止返回值优化 + 有移动构造函数

编译指令：g++ test.cpp --std=c++11 -fno-elide-constructors -o test

```

Test(): this->buf @ 0x7f8951c04b90
GetTemp(): tmp.buf @ 0x7f8951c04b90
Test(Test&&) called. this->buf @ 0x7f8951c04b90
~Test(): this->buf @ 0x0 (tmp被移动后)
Test(Test&&) called. this->buf @ 0x7f8951c04b90 (a = GetTemp())
~Test(): this->buf @ 0x0 (GetTemp返回的临时对象)
main() : a.buf @ 0x7f8951c04b90
Test(const Test&) called. this->buf @ 0x7f8951c04ba0
fun(Test t): t.buf @ 0x7f8951c04ba0
~Test(): this->buf @ 0x7f8951c04ba0
~Test(): this->buf @ 0x7f8951c04b90

```

注意观察：

1. Test a = GetTemp() 调用了移动构造函数（因为 GetTemp() 返回的是右值）
2. fun(a) 调用了拷贝构造函数（因为 a 是左值）
3. 移动构造后，原对象的 buf 变成了 nullptr

情况三：禁止返回值优化 + 删除移动构造函数

编译指令：g++ test.cpp --std=c++11 -fno-elide-constructors -o test

```

Test(): this->buf @ 0x7fabf8c04b50
GetTemp(): tmp.buf @ 0x7fabf8c04b50
Test(const Test&) called. this->buf @ 0x7fabf8c04b60
~Test(): this->buf @ 0x7fabf8c04b50
Test(const Test&) called. this->buf @ 0x7fabf8c04b50
~Test(): this->buf @ 0x7fabf8c04b60
main() : a.buf @ 0x7fabf8c04b50
Test(const Test&) called. this->buf @ 0x7fabf8c04b60
fun(Test t): t.buf @ 0x7fabf8c04b60

```

```
~Test(): this->buf @ 0x7fabf8c04b60
~Test(): this->buf @ 0x7fabf8c04b50
```

没有移动构造函数时，全部使用拷贝构造函数。

什么时候调用移动构造函数？什么时候调用拷贝构造函数？

判断依据：引用的绑定规则

1. 拷贝构造函数的形参类型为 `const ClassName&`（常量左值引用），可以绑定：常量左值、左值、右值
2. 移动构造函数的形参类型为 `ClassName&&`（右值引用），可以绑定：右值
3. **优先级**：当传入右值时，优先匹配右值引用参数的函数

拷贝构造函数的调用时机

1. 用一个类对象/引用/常量引用初始化另一个新的类对象
 - `Test b = a;`（`a` 是左值）
 - `Test b = ref_a;`（`ref_a` 是引用）
2. 以类的对象为函数形参，传入实参为类的对象/引用/常量引用
 - `func(a);`（`a` 是左值）
3. 函数返回类对象（类中未显式定义移动构造函数，不进行返回值优化）

移动构造函数的调用时机

1. 用一个类对象的右值初始化另一个新的类对象
 - `Test b = func(a);`（`func` 返回临时对象，是右值）
 - `Test b = std::move(a);`（`std::move` 把左值转成右值）
2. 以类的对象为函数形参，传入实参为类对象的右值
 - `func(Test());`（临时对象是右值）
 - `func(std::move(a));`（`std::move` 把 `a` 转成右值）
3. 函数返回类对象（类中显式定义移动构造函数，不进行返回值优化）
 - `return Test();` 或 `return tmp;` 均调用移动构造

返回值优化（RVO）

编译器默认会进行返回值优化，条件：

1. `return` 的值类型与函数签名的返回值类型相同
2. `return` 的是一个局部对象

开启 RVO 时，编译器会直接在调用者的栈帧上构造返回值，完全省去拷贝/移动构造函数的调用。

注意：如果你想观察移动构造函数的效果，需要用 `-fno-elide-constructors` 禁用 RVO。

`std::move` 函数

`std::move` 可以将左值转换为右值，从而触发移动构造函数：

```
Test a;
Test b = std::move(a); // 调用移动构造函数
```

`std::move` 本身不做任何操作，只是类型转换。实际的“移动”操作在移动构造函数中实现。

典型应用：高效 swap

```
template <class T>
void swap(T& a, T& b) {
    T tmp(std::move(a)); // 移动构造
    a = std::move(b);     // 移动赋值
    b = std::move(tmp);   // 移动赋值
}
```

避免了三次不必要的拷贝操作！

构造函数综合实例

写出以下代码的运行结果：

```
#include <iostream>
class Test {
public:
    Test() {
        printf("Test()\n");
    } //默认构造函数
    ~Test() {
        printf("~Test()\n");
    } //析构函数
    Test(const Test &con) {
        printf("Test(const Test &con)\n");
    } //拷贝构造函数
    Test(Test &&con) {
        printf("Test(Test &&con)\n");
    } //移动构造函数
};
Test func(Test a) {
    return Test();
}
int main() {
    Test a;
    Test b = func(a);
    return 0;
}
```

编译指令：g++ test.cpp --std=c++11 -fno-elide-constructors

答案分析：用 (1+) 和 (1-) 这样的形式来对应类的构造和析构。

```
Test()                //(1+) 执行 Test a;
Test(const Test &con)  //(2+) Test b = func(a);
                        //      func(a)传参调用拷贝构造函数
Test()                //(3+) return Test();
                        //      Test() 对应的构造函数
Test(Test &&con)        //(4+) return Test();
                        //      为了传值调用的移动构造函数
~Test()               //(3-) return Test();
                        //      Test() 对应的析构函数
Test(Test &&con)        //(5+) Test b = func(a);
                        //      给 b 传值时调用的移动构造函数
~Test()               //(4-) Test b = func(a);
                        //      完成赋值后 func(a) 返回值对应的析构函数
~Test()               //(2-) Test b = func(a);
                        //      参数释放对应的析构函数
~Test()               //(5-) 析构 b
~Test()               //(1-) 析构 a
```

详细步骤解析：

步骤	代码	说明
1	Test a;	构造 a，调用默认构造函数
2	func(a) 传参	拷贝构造函数，将 a 拷贝给参数对象
3	return Test();	构造临时对象，调用默认构造函数
4	return Test();	移动构造函数，将临时对象移动给返回值

5	临时对象析构	步骤 3 创建的临时对象析构
6	Test b = ...	移动构造函数，将返回值移动给 b
7	返回值析构	步骤 4 创建的返回值对象析构
8	参数对象析构	步骤 2 创建的参数对象析构
9	main 结束	析构 b，然后析构 a

关键点:

1. 函数参数按值传递时，会调用拷贝构造函数
2. 函数返回临时对象时，会调用移动构造函数（如果有定义）
3. 析构顺序与构造顺序相反（栈的 LIFO 特性）

多选题

Test 类中显式声明了三类构造函数:

- **普通构造函数**: Test(int val)
- **移动构造函数**: Test(Test&& t)
- **拷贝构造函数**: Test(const Test& t)

现给出一段测试代码，下列描述错误的是:

```
Test F(Test&& a){
    Test b = a;           // (2)
    const Test& c = b;    // (3)
    return c;             // (4)
}
int main(){
    Test A = F(1); // (1)
    return 0;
}
```

选项:

- **A**: (1)处将 1 传入 F 时会调用普通构造函数 Test(int val) 以构建临时对象。
- **B**: (2)处调用移动构造函数。
- **C**: (3)处调用拷贝构造函数。
- **D**: (4)处返回局部变量的引用，可能会为程序带来潜在的风险。

答案: BCD

(1) Test A = F(1); — 选项 A（正确描述）

F 的参数类型为 Test&&，传入整数字面量 1 时，编译器需要先隐式构造一个 Test 类型的临时对象，因此会调用 Test(int val) 普通构造函数，再将该临时对象绑定到右值引用参数 a。

 **A 描述正确**

(2) Test b = a; — 选项 B（错误描述）

关键点: a 的类型虽然是 Test&&，但**具名的右值引用在函数体内是左值**（named rvalue reference is lvalue）。因此 Test b = a 匹配 Test(const Test& t)，调用的是**拷贝构造函数**，而非移动构造函数。若想触发移动构造，应写:

```
Test b = std::move(a);
```

 **B 描述错误**: 实际调用的是拷贝构造函数

(3) const Test& c = b; — 选项 C（错误描述）

此处仅将 c 绑定为 b 的 const 引用，**不构造任何新对象**，不调用任何构造函数。

✗ C 描述错误：没有调用任何构造函数

(4) return c; — 选项 D (错误描述)

函数返回类型是 Test (按值返回)，而非 Test&。return c 会以 c 为源**拷贝构造**一个新的返回值对象，返回的是值副本，不是对局部变量的引用，不存在悬空引用风险。

✗ D 描述错误：返回的是值，不是引用，无悬空风险

十、拷贝赋值运算符与移动赋值运算符

拷贝赋值运算符

已定义的对象之间相互赋值，通过调用对象的“拷贝赋值运算符函数”来实现：

```
ClassName& operator= (const ClassName& right) {  
    if (this != &right) { // 避免自己赋值给自己  
        // 将right对象中的内容拷贝到当前对象中...  
    }  
    return *this;  
}
```

注意区分下面两种代码：

```
ClassName a;  
ClassName b;  
a = b;          // 赋值运算符: a 和 b 都已存在  
ClassName a = b; // 拷贝构造函数: a 还不存在
```

拷贝赋值运算符实例

```
class Test {  
public:  
    int *buf;  
    Test() : buf(new int[10]) {}  
    ~Test() { if (buf) delete[] buf; }  
  
    Test& operator= (const Test& right) {  
        if (this == &right)  
            cout << "same obj!\n";  
        else {  
            for(int i=0; i<10; i++)  
                buf[i] = right.buf[i]; // 拷贝数据  
            cout << "operator=(const Test&) called.\n";  
        }  
        return *this;  
    }  
};
```

重要规则：赋值重载函数必须是类的**非静态成员函数** (non-static member function)，不能是友元函数。

移动赋值运算符

和移动构造函数原理类似，直接“偷走”资源的指针：

```
Test& operator= (Test&& right) {  
    if (this == &right)
```

```

        cout << "same obj!\n";
    else {
        this->buf = right.buf; // 直接赋值地址
        right.buf = nullptr;   // 置空, 防止重复释放
        cout << "operator=(Test&&) called.\n";
    }
    return *this;
}

```

配合 `std::move` 实现高效 swap:

```

template <class T>
void swap(T& a, T& b) {
    T tmp(std::move(a)); // 第一行调用移动构造函数
    a = std::move(b);    // std::move 的结果为右值引用
    b = std::move(tmp);  // 后两行均调用移动赋值运算符
}

```

拷贝/移动赋值运算符的调用时机

判断依据: 引用的绑定规则

1. 拷贝赋值运算符的形参类型为 `const ClassName&` (常量左值引用), 可以绑定: 常量左值、左值、右值
2. 移动赋值运算符的形参类型为 `ClassName&&` (右值引用), 可以绑定: 右值
3. **优先级:** 当赋值运算符右侧为右值时, 优先匹配右值引用参数的函数

场景	调用的运算符	原因
<code>a = b</code> (b 是左值)	拷贝赋值运算符	左值绑定 <code>const &</code>
<code>a = std::move(b)</code>	移动赋值运算符	右值绑定 <code>&&</code>
<code>a = func()</code> (返回临时对象)	移动赋值运算符	右值绑定 <code>&&</code>
<code>a = Test()</code>	移动赋值运算符	临时对象是右值

编译器自动合成的函数/运算符

类中特殊的成员函数/运算符, 即使用户不显式定义, 编译器也会根据需要自动合成:

类型	说明
默认构造函数	无参构造函数
拷贝构造函数	<code>const ClassName&</code> 参数
移动构造函数	<code>ClassName&&</code> 参数 (C++11 起)
拷贝赋值运算符	<code>operator=(const ClassName&)</code>
移动赋值运算符	<code>operator=(ClassName&&)</code> (C++11 起)
析构函数	<code>~ClassName()</code>

注意: 如果你定义了任何构造函数, 编译器就不会自动生成默认构造函数了! 但拷贝/移动构造函数和赋值运算符仍会自动生成 (除非显式删除)。

十一、类型转换

类型转换概述

当编译器发现表达式和函数调用所需的数据类型和实际类型不同时, 便会进行自动类型转换。

```

void print(int d) { }
int main() {
    print(3.5); // double -> int, 自动类型转换
    print('c'); // char -> int, 自动类型转换
}

```

```
    return 0;
}
```

自动类型转换可通过两种方式实现：

1. 定义**类型转换运算符**（在源类中）
2. 定义**类型转换构造函数**（在目标类中）

方法一：类型转换运算符

在**源类**中定义“目标类型转换运算符”：

```
#include <iostream>
using namespace std;
class Dst { // 目标类 Destination
public:
    Dst() { cout << "Dst::Dst()" << endl; }
};
class Src { // 源类 Source
public:
    Src() { cout << "Src::Src()" << endl; }
    operator Dst() const {
        cout << "Src::operator Dst() called" << endl;
        return Dst();
    }
};
```

语法说明：

1. operator 目标类型() 是特殊的成员函数
2. **不需要指定返回类型**，因为 operator Dst() 已经指明了返回 Dst
3. 必须是成员函数，不能是友元

方法二：类型转换构造函数

在**目标类**中定义“源类对象作参数的构造函数”：

```
#include <iostream>
using namespace std;
class Src; // 前置类型声明，因为在 Dst 中要用到 Src 类
class Dst {
public:
    Dst() { cout << "Dst::Dst()" << endl; }
    Dst(const Src& s) { // 源类对象作参数的构造函数
        cout << "Dst::Dst(const Src&)" << endl;
    }
};
class Src {
public:
    Src() { cout << "Src::Src()" << endl; }
};
```

两种方法的使用

两种方法任选一种即可：

```
void Transform(Dst d) { }
int main() {
    Src s;
    Dst d1(s); // 显式初始化，两种方法都支持
    Dst d2 = s; // 隐式转换，两种方法都支持
}
```

```

Transform(s); // 隐式转换, 两种方法都支持
return 0;
}

```

警告：两种自动类型转换的方法**不能同时使用**，否则会产生二义性，编译器不知道选择哪个。使用时任选其中一种！

类型转换运算符：错误示例

```

class SmallInt;
operator int(SmallInt&); // 错误: 不是成员函数
class SmallInt {
public:
    int operator int() const;    // 错误: 不能指定返回类型
    operator int(int = 0) const; // 错误: 参数列表不为空
    operator int*() const { return 42; } // 错误: 42不是合法指针
};

```

错误分析：

1. 类型转换运算符必须是成员函数，不能是全局函数
2. 不能指定返回类型（operator int() 已经隐含返回 int）
3. 参数列表必须为空（转换运算符不接收额外参数）
4. 必须返回正确类型的值

正确写法：

```

class SmallInt {
public:
    operator int() const { return val; } // 正确
    operator int*() const { return &val; } // 正确
private:
    int val;
};

```

类型转换实例

```

class SmallInt {
public:
    // 构造函数: 从 int 转换为 SmallInt
    SmallInt(int i=0): val(i) {
        cout << "SmallInt_Init" << endl;
    }
    // 转换运算符: 从 SmallInt 转换为 int
    operator int() const {
        cout << "Int_Transform" << endl;
        return val;
    }
    void print() { cout << val << endl; }
private:
    size_t val;
};

int main() {
    SmallInt si;
    si = 4.10;
    si = si + 3;
    si.print();
}

```

```

    return 0;
}

```

输出:

```

SmallInt_Init    // SmallInt si, 调用构造函数
SmallInt_Init    // si = 4.10, double->int->SmallInt
Int_Transform    // si + 3, SmallInt->int
SmallInt_Init    // si = si + 3, int->SmallInt
7                // si.print()

```

详细执行流程:

语句	转换过程	输出
SmallInt si;	默认构造函数, val=0	SmallInt_Init
si = 4.10;	4.10→4(double→int)→SmallInt(4)→赋值	SmallInt_Init
si + 3	si→int (调用 operator int()) → 7	Int_Transform
si = 7	7→SmallInt(7)→赋值	SmallInt_Init
si.print()	输出 val	7

思考题解答: 为什么 si = si + 3 不是把 3 转换为 SmallInt 再加呢?

因为 operator+ 是内置的整数加法运算符, 不是 SmallInt 的成员函数!

执行过程:

1. 编译器看到 si + 3, 首先检查 SmallInt 是否有 operator+
2. 没有找到, 于是尝试将 si 转换为能和 int 相加的类型
3. 发现 operator int(), 将 si 转换为 int, 得到 0 + 3 = 3 (初始 val=0)
4. 然后赋值 si = 3, 调用构造函数将 3 转换为 SmallInt

如果想让 3 转换为 SmallInt 再加, 需要重载 operator+:

```

SmallInt operator+(const SmallInt& other) const {
    return SmallInt(val + other.val);
}

```

这样 si + 3 就会: 3→SmallInt(3)→si.operator+(SmallInt(3))

禁止自动类型转换

如果用 explicit 修饰类型转换运算符或类型转换构造函数, 则相应的类型转换必须**显式**进行:

```

// 方法一: explicit 修饰转换运算符
explicit operator Dst() const;

// 方法二: explicit 修饰构造函数
explicit Dst(const Src& s);

```

使用 explicit 后:

```

int main() {
    Src s;
    Dst d1(s);           // ✓ 可以执行, 被认为是显式初始化
    // Dst d2 = s;       // ✗ 错误, 隐式转换被禁止
    // Transform(s);     // ✗ 错误, 隐式转换被禁止
}

```

```

Dst d2 = static_cast<Dst>(s); // ✓ 显式转换
Transform(static_cast<Dst>(s)); // ✓ 显式转换
return 0;
}

```

强制类型转换

C++ 提供四种强制类型转换：

关键字	功能	应用场景
const_cast	去除 const 或 volatile 属性	修改常量指针的指向内容
static_cast	静态类型转换，类似 C 风格	基本类型转换、void 转其他指针
dynamic_cast	动态类型转换（运行时检查）	多态类型间的安全下行转换
reinterpret_cast	重新解释类型（不进行二进制转换）	指针与整数互转、不相关指针互转

详细说明：

1. const_cast：唯一能去除常量性的转换

```

const int* p = &a;
int* q = const_cast<int*>(p); // 去除 const

```

2. static_cast：最常用的转换，编译时检查

```

double d = 3.14;
int i = static_cast<int>(d); // double -> int
void* p = malloc(100);
int* q = static_cast<int*>(p); // void* -> int*

```

3. dynamic_cast：安全的下行转换，需要多态支持（有虚函数）

```

Base* b = new Derived;
Derived* d = dynamic_cast<Derived*>(b); // 安全转换
// 如果转换失败，指针返回 nullptr，引用抛出异常

```

4. reinterpret_cast：最危险的转换，直接重新解释内存

```

int* p = new int(42);
long addr = reinterpret_cast<long>(p); // 指针 -> 整数
int* q = reinterpret_cast<int*>(addr); // 整数 -> 指针

```

强制类型转换示例

```

int main() {
    Src s;
    Dst d1(s); // 显式初始化
    Dst d2 = static_cast<Dst>(s); // 显式转换
    Transform(static_cast<Dst>(s)); // 显式转换
    return 0;
}

```

陆 组合与继承 2026-04-07

一、对象(类)之间的关系

思考：这些是什么关系？

- 智能体：感知模块，思考模块，执行模块

- 智能体：科研智能体，编程智能体，写作智能体

1. **has-a**：感知，思考，执行模块是智能体的组成部分（“整体-部分”）
2. **is-a**：科研，编程，写作智能体是具有特殊能力的智能体（“一般-特殊”）

二、组合

has-a：如果对象 a 是对象 b 的一个组成部分，则称 b 为 a 的整体对象，a 为 b 的部分对象。并把 b 和 a 之间的关系，称为“整体-部分”关系（也可称为“组合”或“has-a”关系）。

程序设计反映对客观世界的认知习惯。

对象组合的两种实现方法：

1. 已有类的对象作为新类的**公有**数据成员，这样通过允许直接访问子对象而“提供”旧类接口
2. 已有类的对象作为新类的**私有**数据成员。新类可以调整旧类的对外接口，可以不使用旧类原有的接口（相当于对接口作了转换）

对象组合示例

```
#include <iostream>
#include <string>
using namespace std;
class Perception{
    string _prompt;
public:
    void set(const string& prompt){_prompt=prompt;}
};
class Action{
    string _prompt;
public:
    void set(const string& prompt){_prompt=prompt;}
};
class Agent{
private:
    Perception p;
public:
    Action a; // 公有成员，直接访问其接口
    void setPerception(const string& prompt){p.set(prompt);} // 提供私有成员的访问接口
};
int main(){
    Agent agent;
    string prompt = "Hello, World";
    agent.a.set(prompt);
    agent.setPerception(prompt);
    return 0;
}
```

子对象构造

子对象构造时若需要参数，则应在当前类的构造函数的初始化列表中进行。若使用默认构造函数来构造子对象，则不用做任何处理。

对象构造与析构函数的次序：

1. 先完成子对象构造，再完成当前对象构造
2. 子对象构造的次序仅由在类中声明的次序所决定
3. 析构函数的次序与构造函数相反

对象组合示例：构造与析构

```
#include <iostream>
using namespace std;
class S1 { //Single1类别
```

```

    int ID;
public:
    S1(int id) : ID(id) { cout << "S1(int)" << endl; }
    ~S1() { cout << "~S1()" << endl; }
};
class S2 { //Single2类别
public:
    S2() { cout << "S2()" << endl; }
    ~S2() { cout << "~S2()" << endl; }
};
class C3 { //Composite3类别
    int num;
    S1 sub_obj1; // 构造函数带参数
    S2 sub_obj2; // 构造函数不带参数
public:
    C3() : num(0), sub_obj1(123) { cout << "C3()" << endl; }
    C3(int n) : num(n), sub_obj1(123) { cout << "C3(int)" << endl; }
    C3(int n, int k) : num(n), sub_obj1(k) { cout << "C3(int, int)" << endl; }
    ~C3() { cout << "~C3()" << endl; }
};
int main(){
    C3 a, b(1), c(2), d(3, 4);
    return 0;
}

```

对象组合运行结果

```

S1(int)
S2()
C3()
S1(int)
S2()
C3(int)
S1(int)
S2()
C3(int)
S1(int)
S2()
C3(int, int)
~C3()
~S2()
~S1()
~C3()
~S2()
~S1()
~C3()
~S2()
~S1()
~C3()
~S2()
~S1()

```

晚构造，先析构！

隐式定义的拷贝构造与赋值运算

回忆：如果调用拷贝构造函数且没有给类显式定义拷贝构造函数，编译器将提供“隐式定义的拷贝构造函数”。该函数的功能为：

1. 递归调用所有子对象的拷贝构造函数

2. 对于基础类型，采用位拷贝
3. 赋值运算的默认操作类似

对象组合示例：拷贝与赋值

```
#include <iostream>
using namespace std;
class C1{
public:
    int i;
    C1(int n):i(n){}
    C1(const C1 &other) // 显式定义拷贝构造函数
    {i=other.i; cout << "C1(const C1 &other)" << endl;}
};
class C2{
public:
    int j;
    C2(int n):j(n){}
    C2& operator= (const C2& right){ // 显式定义赋值运算符
        if(this != &right){
            j = right.j;
            cout << "operator=(const C2&)" << endl;
        }
        return *this;
    }
};
```

```
class C3{
public:
    C1 c1;
    C2 c2;
    C3():c1(0), c2(0){}
    C3(int i, int j):c1(i), c2(j){}
    void print(){cout << "c1.i = " << c1.i << " c2.j = " << c2.j << endl;}
};
int main(){
    C3 a(1, 2);
    C3 b(a); // C1执行显式定义的拷贝构造, C2执行隐式定义的拷贝构造
    cout << "b: ";
    b.print();
    cout << endl;
    C3 c;
    cout << "c: ";
    c.print();
    c = a; // C1执行隐式定义的拷贝赋值, C2执行显式定义的拷贝赋值
    cout << "c: ";
    c.print();
    return 0;
}
```

运行结果:

```
C1(const C1 &other)
b: c1.i = 1 c2.j = 2
c: c1.i = 0 c2.j = 0
operator=(const C2&)
c: c1.i = 1 c2.j = 2
```

三、继承

is-a: “一般-特殊”结构, 也称“分类结构”, 是由一组具有“一般-特殊”关系的类所组成的结构。

1. 如果类 A 具有类 B 全部的属性和服务, 而且具有自己特有的某些属性或服务, 则称 A 为 B 的特殊类, B 为 A 的一般类。
2. 如果类 A 的全部对象都是类 B 的对象, 而且类 B 中存在不属于类 A 的对象, 则 A 是 B 的特殊类, B 为 A 的一般类。

C++ 使用继承来表达类间的“一般-特殊结构”。例如“编程智能体”继承“智能体”。

基本概念

1. 被继承的已有类, 被称为**基类** (base class), 也称“父类”。
2. 通过继承得到的新类, 被称为**派生类** (derived class), 也称“子类”、“扩展类”。

常见的继承方式:

1. `class Derived : public Base { ... };` — 公有继承
2. `class Derived : [private] Base { .. };` — 私有继承 (缺省)
3. `class Derived : protected Base { ... };` — 保护继承 (很少使用)

什么不能被继承?

1. **构造函数**: 创建派生类对象时, 必须调用派生类的构造函数, 派生类构造函数调用基类的构造函数, 以创建派生对象的基类部分。C++11 新增了继承构造函数的机制 (使用 `using`), 但默认不继承。
2. **析构函数**: 释放对象时, 先调用派生类析构函数, 再调用基类析构函数。
3. **赋值运算符**: 编译器不会继承基类的赋值运算符 (参数为基类); 但会自动合成隐式定义的赋值运算符 (参数为派生类), 其功能为调用基类的赋值运算符。
4. **友元函数**: 不是类成员, 不能被继承。

继承示例

```
#include <iostream>
using namespace std;
class Base{
public:
    int k = 0;
    void f(){cout << "Base::f()" << endl;}
    Base & operator= (const Base &right){
        if(this != &right){
            k = right.k;
            cout << "operator= (const Base &right)" << endl;
        }
        return *this;
    }
};
class Derive: public Base{};
int main(){
    Derive d, d2;
    cout << d.k << endl; //Base数据成员被继承
    d.f(); //Base::f()被继承
    Base e;
    //d = e; //编译错误, Base的赋值运算符不被继承
    d = d2; //调用隐式定义的赋值运算符
    return 0;
}
```

运行结果:

```
0
Base::f()
operator= (const Base &right)
```

派生类对象的构造与析构过程

基类中的数据成员，通过继承成为派生类对象的一部分，需要在构造派生类对象的过程中调用基类构造函数来正确初始化。

1. 若没有显式调用，则编译器会自动调用基类的默认构造函数。
2. 若想要显式调用，则只能在派生类构造函数的**初始化成员列表**中进行，既可以调用基类中不带参数的默认构造函数，也可以调用合适的带参数的其他构造函数。

先执行基类的构造函数来初始化继承来的数据，再执行派生类的构造函数。

对象析构时，**先执行派生类析构函数，再执行由编译器自动调用的基类的析构函数。**

调用基类构造函数：隐式调用默认构造

```
class Base
{
    int data;
public:
    Base() : data(0) { cout << "Base::Base(" << data << ")\n"; }
    Base(int i) : data(i) { cout << "Base::Base(" << data << ")\n"; }
};
class Derive : public Base {
public:
    Derive() { cout << "Derive::Derive()" << endl; }
    // 无显式调用基类构造函数，则调用基类默认构造函数
};
int main() {
    Derive obj;
    return 0;
}
```

运行结果：

```
Base::Base(0)
Derive::Derive()
```

调用基类构造函数：显式调用

若想要显式调用，则只能在派生类构造函数的初始化成员列表中进行。

```
class Base
{
    int data;
public:
    Base() : data(0) { cout << "Base::Base(" << data << ")\n"; }
    Base(int i) : data(i) { cout << "Base::Base(" << data << ")\n"; }
};
class Derive : public Base {
public:
    Derive(int i) : Base(i) { cout << "Derive::Derive()" << endl; }
    // 显式调用基类构造函数
};
int main() {
    Derive obj(356);
    return 0;
}
```

运行结果：

```
Base::Base(356)
Derive::Derive()
```

继承基类构造函数（using）

在派生类中使用 `using Base::Base;` 来继承基类构造函数，相当于给派生类“定义”了相应参数的构造函数。

```
class Base
{
    int data;
public:
    Base(int i) : data(i) { cout << "Base::Base(" << i << ")\n"; }
};
class Derive : public Base {
public:
    using Base::Base; //相当于 Derive(int i):Base(i){};
};
int main() {
    Derive obj(356);
    return 0;
}
```

运行结果：

```
Base::Base(356)
```

继承基类构造函数（多个构造函数）

当基类存在多个构造函数时，使用 `using` 会给派生类自动构造多个相应的构造函数。

```
class Base
{
    int data;
public:
    Base(int i) : data(i) { cout << "Base::Base(" << i << ")\n"; }
    Base(int i, int j)
    { cout << "Base::Base(" << i << "," << j << ")\n"; }
};
class Derive : public Base {
public:
    using Base::Base; //相当于 Derive(int i):Base(i){};
                        //加上 Derive(int i, int j):Base(i, j){};
};
int main() {
    Derive obj1(356);
    Derive obj2(356, 789);
    return 0;
}
```

运行结果：

```
Base::Base(356)
Base::Base(356,789)
```

继承基类构造函数（注意事项）

1. 如果基类的某个构造函数被声明为私有成员函数，则不能在派生类中声明继承该构造函数。
2. 如果派生类使用了继承构造函数，编译器就不会再为派生类生成隐式定义的默认构造函数。

如何选择继承方式?

public 继承

1. 基类中公有成员仍能在派生类中保持公有。原接口可沿用。最常用。
2. is-a: 基类对象能使用的地方, 派生类对象也能使用。

private 继承

1. is-implementing-in-terms-of (照此实现): 用基类接口实现派生类功能。移除了 is-a 关系。
2. 通常不使用, 用组合替代。可用于隐藏/公开基类的部分接口。公开方法: using 关键字。

成员访问权限

1. 基类中的**私有成员**, 不允许在派生类成员函数中访问, 也不允许派生类的对象访问它们。真正体现“基类私有”, 对派生类也不开放其权限!
2. 基类中的**公有成员**:
 - 允许在派生类成员函数中被访问
 - 若是使用 public 继承方式, 则成为派生类公有成员, 可以被派生类的对象访问
 - 若是使用 private/protected 继承方式, 则成为派生类私有/保护成员, 不能被派生类的对象访问。若想让某成员能被派生类的对象访问, 可在派生类 public 部分用关键字 using 声明它的名字
3. 基类中的**保护成员**:
 - 保护成员允许在派生类成员函数中被访问, 但不能被外部函数访问

公有继承: 基类公有成员的访问

```
#include <iostream>
using namespace std;
class Base {
public:
    void baseFunc() { cout << "in Base::baseFunc()..." << endl; }
};
class Derive1: public Base {}; // public继承
int main() {
    Derive1 obj1;
    cout << "calling obj1.baseFunc()..." << endl;
    obj1.baseFunc(); // 基类接口成为派生类接口的一部分, 派生类对象可调用
    return 0;
}
```

私有继承: 基类公有成员的访问

```
#include <iostream>
using namespace std;
class Base {
public:
    void baseFunc() { cout << "in Base::baseFunc()..." << endl; }
};
class Derive2: private Base {
// 私有继承, is-implementing-in-terms-of: 用基类接口实现派生类功能
public:
    void deriveFunc() {
        cout << "in Derive2::deriveFunc(), calling Base::baseFunc()..." << endl;
        baseFunc(); // 私有继承时, 基类接口在派生类成员函数中可以使用
    }
};
int main() {
    Derive2 obj2;
    cout << "calling obj2.deriveFunc()..." << endl;
    obj2.deriveFunc();
    //obj2.baseFunc(); ERROR: 基类接口不允许从派生类对象调用
}
```

```

    return 0;
}

```

私有继承：用 using 打开基类公有成员的访问权限

```

#include <iostream>
using namespace std;
class Base {
public:
    void baseFunc() { cout << "in Base::baseFunc()..." << endl; }
};
class Derive3: private Base {
public:
    using Base::baseFunc; // 私有继承时，在派生类public部分声明基类成员名字
};
int main() {
    Derive3 obj3;
    cout << "calling obj3.baseFunc()..." << endl;
    obj3.baseFunc(); // 基类接口在派生类public部分声明，则派生类对象可调用
    return 0;
}

```

私有继承：基类私有、保护成员访问

```

#include <iostream>
using namespace std;
class Base{
private:
    int a{0};
protected:
    int b{0};
};
class Derive : private Base{
public:
    void getA(){cout<<a<<endl;} // 编译错误，不可访问基类中私有成员
    void getB(){cout<<b<<endl;} // 可以访问基类中保护成员
};
int main()
{
    Derive d;
    d.getB();
    //cout<<d.b; // 编译错误，派生类对象不可访问基类中保护成员
    return 0;
}

```

```

#include <iostream>
using namespace std;
class Base {
private:
    int data{0};
public:
    int getData(){ return data;}
    void setData(int i){ data=i;}
};
class Derive1 : private Base {
public:
    using Base::getData;
}

```

```
};
int main() {
    Derivel d1;
    cout<<d1.getData();
    //d1.setData(10); // 隐藏了基类的setData函数, 不可访问
    //Base& b = d1; // 不允许私有继承的向上转换
    //b.setData(10); // 否则可以绕过D1, 调用基类的setData函数
    return 0;
}
```

基类成员访问权限与三种继承方式

1. **public 继承**: 基类的公有成员、保护成员、私有成员作为派生类的成员时, 都保持原有的状态。
2. **private 继承**: 基类的公有成员、保护成员、私有成员作为派生类的成员时, 都作为私有成员。
3. **protected 继承**: 基类的公有成员、保护成员作为派生类的成员时, 都成为保护成员, 基类的私有成员仍然是私有的。

继承权限表:

派生类成员函数能否访问基类成员			
	public 继承	private 继承	protected 继承
基类 public 成员	✓	✓	✓
基类 private 成员	×	×	×
基类 protected 成员	✓	✓	✓

基类成员在派生类中的成员类型 / 派生类对象能否访问			
	public 继承	private 继承	protected 继承
基类 public 成员	public / ✓	private / ×	protected / ×
基类 private 成员	private / ×	private / ×	private / ×
基类 protected 成员	protected / ×	private / ×	protected / ×

类似集合交运算: 成员类型与继承类型之间取交。Order: public > protected > private。

组合与继承的对比

优点: 支持增量开发。允许引入新代码而不影响已有代码正确性。

相似:

1. 实现代码重用
2. 将子对象引入新类
3. 使用构造函数的初始化成员列表初始化

不同:

1. 组合:
 - 嵌入一个对象以实现新类的功能
 - has-a 关系
2. 继承:
 - 沿用已存在的类提供的接口
 - public 继承: is-a
 - private 继承: is-implementing-in-terms-of

组合示例: has-a

```
#include <iostream>
using namespace std;
class Perception{
public:
```

```

    void init(){ cout<<"Perception::init"<<endl; }
};
class Action{
public:
    void init(){ cout<<"Action::init"<<endl; }
    void stop(){}
};
class Agent{
public:
    Perception perception;
    Action action;
};
int main()
{
    Agent agent;
    agent.perception.init();
    agent.action.init();
    return 0;
}

```

运行结果:

```

Perception::init
Action::init

```

继承示例: is-a

```

#include <iostream>
using namespace std;
class Agent{
public:
    void init(){cout<<"Agent init"<<endl;}
    void act(){cout<<"Agent act"<<endl; }
};
class Coder: public Agent{
public:
    void act(){cout<<"Coder act"<<endl;}
};
int main()
{
    Coder coder;
    coder.init();
    coder.act();
    return 0;
}

```

运行结果:

```

Agent init
Coder act

```

四、重写隐藏与重载

重载 (overload)

1. 目的: 提供同名函数的不同实现, 属于静态多态。
2. 函数名必须相同, 函数参数必须不同, 作用域相同 (如位于同一个类中; 或同名全局函数)。

重写隐藏 (redefining)

1. 目的：在派生类中重新定义基类函数，实现派生类的特殊功能。
2. **屏蔽了基类的所有其它同名函数。**
3. 函数名必须相同，函数参数可以不同。

重写隐藏发生时，基类中该成员函数的其他重载函数都将被屏蔽掉，不能提供给派生类对象使用。

可以在派生类中通过 `using 类名::成员函数名`；在派生类中“恢复”指定的基类成员函数（即去掉屏蔽），使之重新可用。

函数重写隐藏示例

```
#include <iostream>
using namespace std;
class T {};
class Base {
public:
    void f() { cout << "B::f()\n"; }
    void f(int i) { cout << "Base::f(" << i << ")\n"; } // 重载
    void f(double d) { cout << "Base::f(" << d << ")\n"; } // 重载
    void f(T) { cout << "Base::f(T)\n"; } // 重载
};
class Derive : public Base {
public:
    void f(int i) { cout << "Derive::f(" << i << ")\n"; } // 重写隐藏
};
int main() {
    Derive d;
    d.f(10);
    d.f(4.9); // 编译警告。执行自动类型转换。
    //d.f(); // 被屏蔽，编译错误
    //d.f(T()); // 被屏蔽，编译错误
    return 0;
}
```

运行结果：

```
Derive::f(10)
Derive::f(4)
```

4.9 → 4：自动类型转换，因为基类的 `f(double)` 被屏蔽了，只能匹配派生类的 `f(int)`。

恢复基类成员函数示例

使用 `using 基类名::函数名`；恢复基类函数：

```
#include <iostream>
using namespace std;
class T {};
class Base {
public:
    void f() { cout << "Base::f()\n"; }
    void f(int i) { cout << "Base::f(" << i << ")\n"; }
    void f(double d) { cout << "Base::f(" << d << ")\n"; }
    void f(T) { cout << "Base::f(T)\n"; }
};
class Derive : public Base {
public:
    using Base::f;
    void f(int i) { cout << "Derive::f(" << i << ")\n"; }
}
```

```
};
int main() {
    Derive d;
    d.f(10);
    d.f(4.9);
    d.f();
    d.f(T());
    return 0;
}
```

运行结果:

```
Derive::f(10)
Base::f(4.9)
Base::f()
Base::f(T)
```

using 关键字总结

using 关键字可用于:

1. 继承基类构造函数: using Base::Base;
2. 恢复被屏蔽的基类成员函数: using Base::f;
3. 指示命名空间: using namespace std;
4. 将另一个命名空间的成员引入当前命名空间: using std::cout;
5. 定义类型别名: using a = int;

五、多重继承

派生类同时继承多个基类。

应用场景:

```
class File{};
class InputFile: public File{};
class OutputFile: public File{};
class IOFile: public InputFile, public OutputFile{};
```

多重继承的问题

1. **数据存儲**: 如果派生类 D 继承的两个基类 A, B 是同一基类 Base 的不同继承, 则 A, B 中继承自 Base 的数据成员会在 D 有**两份独立的副本**, 可能带来数据冗余。
2. **二义性**: 如果派生类 D 继承的两个基类 A, B 有同名成员 a, 则访问 D 中 a 时, 编译器无法判断要访问的哪一个基类成员。

多重继承示例

```
#include <iostream>
using namespace std;
class Base {
public:
    int a{0};
};
class MiddleA : public Base {
public:
    void addA() { cout << "a=" << ++a << endl; };
    void bar() { cout << "A::bar" << endl; };
};
class MiddleB : public Base {
public:
    void addB() { cout << "a=" << ++a << endl; };
```

```

    void bar() { cout << "B::bar" << endl; };
};
class Derive : public MiddleA, public MiddleB{
};

```

MiddleA 和 MiddleB 各有一份独立的 Base::a，Derive 继承了两份。

```

int main() {
    Derive d;
    d.addA(); // 输出 a=1
    d.addB(); // 仍然输出 a=1 (另一份副本)
    d.addB(); // 输出 a=2
    //cout << d.a; // 编译错误, MiddleA和MiddleB都有成员a
    cout << d.MiddleA::a << endl; // 输出A中的成员a的值 1
    //d.bar(); // 编译错误, MiddleA和MiddleB都有成员函数bar
    cout << d.MiddleB::a << endl; // 输出B中的成员a的值 2
    return 0;
}

```

解决二义性：用 类名::成员名 显式指定访问哪个基类的成员。

六、课后练习

一家工厂生产飞机、汽车和摩托车。一架飞机需要三个轮子和两个机翼；一辆汽车需要四个轮子；一辆摩托车需要两个轮子。

这些交通工具都具有一个 run 函数，其中汽车和摩托车调用时输出“I am running”，但是飞机调用时输出“I am running and flying”。

编写以下几个类：Plane, Motor, Car, Wing, Wheel, Vehicle（交通工具），设计合理的继承、组合关系以及使用函数的继承与重写实现 add_wing, add_wheel, finished 以及 run 函数。

测试代码：

```

#include <iostream>
#include "Car.h"
#include "Plane.h"
#include "Motor.h"
#include "Wing.h"
#include "Wheel.h"
int main() {
    int m;
    std::cin >> m;
    Plane planes = new Plane[100];
    Car cars = new Car[100];
    Motor motors = new Motor[100];
    int i_p = 0, i_c = 0, i_m = 0;
    for (int i = 0; i < m; ++i) {
        int op;
        std::cin >> op;
        if (op == 0) { // plane
            int part;
            std::cin >> part;
            if (part == 0) planes[i_p].add_wing(new Wing());
            else planes[i_p].add_wheel(new Wheel());
            if (planes[i_p].finished()) planes[i_p++].run();
        }
        else if (op == 1) { // car
            cars[i_c].add_wheel(new Wheel());
        }
    }
}

```

```

        if (cars[i_c].finished()) cars[i_c++].run();
    }
    else { // motor
        motors[i_m].add_wheel(new Wheel());
        if (motors[i_m].finished())motors[i_m++].run();
    }
}
return 0;
}

```

输入:

```

0 0
0 0
0 1
0 1
0 1
1
1
2
2
1
1

```

输出:

```

I am running and flying
I am running
I am running

```